B.TECH CSE · MINI PROJECT · AI / ML

CINEMATCH

A Movie Recommendation System using Content-Based Filtering, Collaborative Filtering & Matrix Factorization

Presented by Niraj Singh
UIT Prayagraj

MOTIVATION

TOO MANY MOVIES, NOT ENOUGH TIME

Streaming platforms host tens of thousands of titles. Manually browsing browsing for something worth watching is slow, and generic “top 10” “top 10” lists ignore individual taste.

Decision fatigue

Users spend more time searching than watching.

One-size-fits-all charts

Trending lists don't reflect personal taste.

Cold, disconnected catalogues

Great niche movies stay undiscovered.

50+ titles in this project alone —
imagine an entire streaming catalogue.

OBJECTIVES

WHAT THIS PROJECT SETS OUT TO DO

01

Build a working recommender

Implement three distinct ML techniques end-to-end in Python.

02

Compare approaches

Show how content, collaborative and latent-factor methods differ in practice.

03

Ship a usable product

Wrap the models in a REST API and an interactive web frontend.

04

Keep it reproducible

Self-contained dataset and code — runs locally with one command.

APPROACH

THREE RECOMMENDATION ENGINES, ONE APP

1

Content-Based Filtering

Recommends by item similarity

TF-IDF over genres + cosine similarity between movies.

2

Collaborative Filtering

Recommends by user similarity

Item-based CF over the user-item rating rating matrix.

3

Matrix Factorization

Recommends by latent taste factors

Truncated SVD learns hidden taste dimensions to predict ratings.

TECHNIQUE 01

CONTENT-BASED FILTERING

Each movie's genre string (e.g. “Action|Sci-Fi|Thriller”) is converted into a converted into a TF-IDF vector. Cosine similarity between vectors then then ranks every other movie by how closely its genre profile matches the matches the one the user picked.

- Works with zero rating history — only needs item metadata
- Great for “more like this” style recommendations
- Limitation: can't discover tastes outside a user's known genres

Genres text

TF-IDF vectors

Cosine similarity

Ranked list

TECHNIQUE 02

COLLABORATIVE FILTERING

Built from the user-item rating matrix. For every pair of movies, cosine similarity is computed across how all 200 simulated users rated them. A user's predicted score for an unseen movie is a similarity-weighted average of the movies they already rated.

- Learns from the community's behaviour, not just item metadata
- Can surface unexpected, cross-genre recommendations
- Limitation: needs enough overlapping ratings to work well (cold-start problem)

	M1	M2	M3	M4	M5
U1	5		4		2
U2		3		5	
U3	4		5		1
U4		4		4	

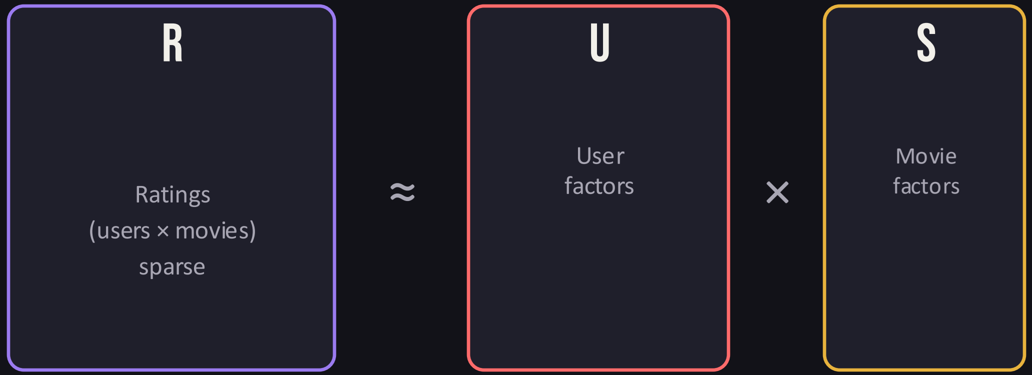
User × Movie rating matrix (sample)

TECHNIQUE 03

MATRIX FACTORIZATION (SVD)

Truncated SVD decomposes the sparse user-item matrix into two smaller matrices of latent “taste factors” — one per user, one per movie. Multiplying them back together reconstructs a dense matrix of predicted ratings for every user-movie pair, even ones with no rating yet.

- Captures subtle taste patterns that aren't explicit genres
- Handles sparse data better than plain collaborative filtering
- Powers the recommendation engine behind most production systems



15 latent factors used in this implementation (scikit-learn TruncatedSVD)

SYSTEM DESIGN

HOW THE PIECES FIT TOGETHER

Browser Frontend
HTML / CSS / JS
Mode tabs + dropdowns

→

Flask REST API
app.py
routes for each engine

→

Recommender Engine
recommender.py
3 ML techniques

→

Dataset
movies.csv
ratings.csv

Request flow

1. User picks a movie/user profile and clicks a button on the frontend
2. Browser calls a REST endpoint, e.g. /api/recommend/collaborative?user_id=7
3. Flask routes the request to the matching function in recommender.py
4. The engine computes similarity/predicted scores and returns ranked JSON
5. The frontend renders results as movie cards with match-score bars

IMPLEMENTATION

TECH STACK

Python 3

Core language for the recommendation engine

pandas

Loading & reshaping the ratings dataset

scikit-learn

TF-IDF, cosine similarity, Truncated SVD

Flask

Lightweight REST API + template rendering

HTML / CSS / JS

Interactive single-page frontend

NumPy

Matrix operations for scoring & ranking

DATA

A SELF-CONTAINED DATASET

Real MovieLens data requires an internet download, so this project generates its own MovieLens-style dataset with the same schema — fully reproducible, offline, and easy to swap for the real thing later.

50 movies across 15+ genres

200 simulated user profiles

~3,250 generated ratings (1–5 scale)

movies.csv (sample)

movieId	title	genres
6	Inception	Action|Sci-Fi|Thriller
7	The Matrix	Action|Sci-Fi
21	Toy Story	Animation|Comedy
41	3 Idiots	Comedy|Drama

ratings.csv schema: userId, movieId, rating (1–5)

LIVE DEMO

CONTENT-BASED RECOMMENDATIONS

Selecting “Inception” surfaces sci-fi/action titles with the closest genre profile.

LIVE DEMO

COLLABORATIVE FILTERING IN ACTION

For User #1 (who rated Queen, Titanic and Interstellar highly), the system surfaces further drama -leaning picks.

EVALUATION

HOW THE THREE TECHNIQUES COMPARE

	Content-Based	Collaborative	Matrix Factorization
Needs rating history?	No	Yes	Yes
Cold-start friendly	Yes	No	Partial
Finds cross-genre picks	No	Yes	Yes
Scales to large catalogues	Good	Moderate	Best
Captures hidden taste patterns	No	Partial	Yes

Qualitative testing across the 200 simulated user profiles confirmed the expected pattern: content-based results stayed within a genre, while collaborative filtering and SVD both surfaced cross-genre titles matching each user's simulated taste bias.

WHAT'S NEXT

FUTURE SCOPE

Hybrid model

Blend content-based + collaborative scores for stronger cold-start handling.

Real MovieLens / TMDB data

Swap the synthetic dataset for a full real-world catalogue.

Deep learning recommenders

Explore neural collaborative filtering / autoencoders.

Deployment

Containerize with Docker and deploy the API + frontend to the cloud.
cloud.

THANK YOU

Questions & Discussion